```
from google.colab import files
uploaded = files.upload()
```

Choose Files Screenshot… 133747.png

Screenshot 2026-08-07 133747.png(image/png) - 142757 bytes, last modified: 8/7/2026 - 100% done

```
Saving Screenshot 2026-08-07 133747.png to Screenshot 2026-08-07 133747.png
```

```
# Feature Matching under Noise and Rotation

import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Read input image
img = cv2.imread("Screenshot 2026-08-07 133747.png", 0)

if img is None:
    print("Image not found! Please ensure 'Screenshot 2026-08-07 133747.png' exists.")
else:
    # 2. Rotate the image
    h, w = img.shape
    center = (w // 2, h // 2)

    M = cv2.getRotationMatrix2D(center, 30, 1.0)
    rotated = cv2.warpAffine(img, M, (w, h))

    # 3. Add Gaussian noise
    noise = np.random.normal(0, 25, rotated.shape)
    noisy_rotated = rotated + noise
    noisy_rotated = np.clip(noisy_rotated, 0, 255).astype(np.uint8)

    # 4. Create ORB detector
    orb = cv2.ORB_create(nfeatures=1000)

    # Features from original image
```

```
kp1, des1 = orb.detectAndCompute(img, None)

# Features from transformed image
kp2, des2 = orb.detectAndCompute(noisy_rotated, None)

print("Original Features:", len(kp1))
print("Transformed Features:", len(kp2))

# 5. Feature matching using BFMatcher
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

matches = bf.match(des1, des2)

# Sort matches by distance
matches = sorted(matches, key=lambda x: x.distance)

# 6. Display number of matches
print("Total Matches:", len(matches))

# 7. Display best matches
result = cv2.drawMatches(
    img, kp1,
    noisy_rotated, kp2,
    matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# 8. Display images
plt.figure(figsize=(14, 6))
plt.imshow(result, cmap="gray")
plt.title("ORB Feature Matching under Noise and Rotation")
plt.axis("off")
plt.show()

# 9. Calculate average matching distance
if len(matches) > 0:
    avg_distance = np.mean([m.distance for m in matches])
```

```
        print("Average Matching Distance:", round(avg_distance, 2))

    # 10. Display original and transformed images
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(img, cmap="gray")
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(noisy_rotated, cmap="gray")
    plt.title("Rotated + Noisy Image")
    plt.axis("off")

    plt.show()
```

```
Original Features: 869
Transformed Features: 854
Total Matches: 395
```